

Lab 9 – Microprocessor Design

Participants

- | | | |
|-----------------------|---|---------|
| 1) Mallikarachchi S.D | - | 210364F |
| 2) Wickramasinghe J.J | - | 210706H |

Lab Task

Design a 4-bit processor capable of executing 4 instructions with the use of new and previously designed components. Verify the functionality via simulation and on the Basys3 board

Assembly Program Code

```
"101110000001", --0 Move 1 to R7
"100100000010", --1 Move 2 to R2
"100110000011", --2 Move 3 to R3
"001110100000", --3 R7 <= R7 + R2
"001110110000", --4 R7 <= R7 + R3
"010110000000", --5 R3 <= -R3
"111110000101", --6 Jump to instruction 5 if Reg7 is 0
"110000000110"  --7 Jump to instruction 6 if Reg0 is 0
```

Program Loop

1. MOVI R ₇ , 1	R ₇ ← 1
2. MOVI R ₂ , 2	R ₂ ← 2
3. MOVI R ₃ , 3	R ₃ ← 3
4. ADD R ₇ , R ₂	R ₇ ← 3 = 1 + 2
5. ADD R ₇ , R ₃	R ₇ ← 6 = 3 + 3
6. NEG R ₃	R ₃ ← (-3)
7. JZR R ₇ , 5	Jump to instruction 5 if Reg7 is 0
8. JZR R ₀ , 6	Jump to instruction 6 if Reg0 is 0

VHDL Code

Nanoprocessor

```
entity NP is
    Port ( Clk : in STD_LOGIC;
          Res : in STD_LOGIC;
          Overflow_led : out STD_LOGIC;
          Zero_led : out STD_LOGIC;
          Reg_7 : out std_logic_vector(3 downto 0);
          --Reg_1 : out std_logic_vector(3 downto 0);
          Seg7_Anode : out STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
          Seg7_Out : out STD_LOGIC_VECTOR(6 downto 0) := (others => '0')
    );
end NP;

architecture Behavioral of NP is
    component InsDecoder
    Port ( -- the instruction from the PC register address
          I_Instruction : in STD_LOGIC_VECTOR (11 downto 0);
          -- for the JZR instruction
          I_Jump_Check : in STD_LOGIC_VECTOR (3 downto 0);
          I_Clk : in STD_LOGIC;
          I_Reset: in STD_LOGIC;
          -- enabling the relevant register in register bank
          O_Reg_Enable : out STD_LOGIC_VECTOR (2 downto 0);
          -- selecting the relevant register input A for Add/Sub unit
          O_Reg_Sel_A : out STD_LOGIC_VECTOR (2 downto 0);
          -- selecting the relevant register input B for Add/Sub unit
          O_Reg_Sel_B : out STD_LOGIC_VECTOR (2 downto 0);
          -- enable writing to PC register
          O_EN_PC : out STD_LOGIC;
          -- enable writing to register bank registers
          O_EN_Store : out STD_LOGIC;
          -- add or sub for Add/Sub unit (1 for subtraction)
          O_Add_Sub_Sel : out STD_LOGIC;
          -- to determine if PC is incremented normally or by address from JZR
          instruction (1 to jump)
          O_Jump_Flag : out STD_LOGIC;
          -- jump address from JZR instruction
          O_Jump_Addr : out STD_LOGIC_VECTOR (2 downto 0);
          -- load immediate value or add/sub result (0 for imm, 1 for add/sub)
          O_Load_Sel : out STD_LOGIC;
          -- immediate value passed from instruction
```

```

O_Imm_Val : out STD_LOGIC_VECTOR (3 downto 0));
end component;

component PC
Port ( In_MUX2_3 : in STD_LOGIC_VECTOR (2 downto 0);
      --EN : in STD_LOGIC;
      Clk : in STD_LOGIC;
      --Inc: in STD_LOGIC;
      Res: in STD_LOGIC;
      Out_PC : inout STD_LOGIC_VECTOR (2 downto 0));
end component;

component PROM
Port ( address_new : in STD_LOGIC_VECTOR (2 downto 0);
      data_new : out STD_LOGIC_VECTOR (11 downto 0));
end component;

component RegBank
Port ( I_Reg_Enable : in STD_LOGIC_VECTOR (2 downto 0);
      I_Data_In : in STD_LOGIC_VECTOR (3 downto 0);
      I_Reset : in STD_LOGIC;
      I_En_Store : in STD_LOGIC;
      I_Clk : in STD_LOGIC;
      Data_out_0:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_1:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_2:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_3:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_4:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_5:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_6:out STD_LOGIC_VECTOR (3 downto 0);
      Data_out_7:out STD_LOGIC_VECTOR (3 downto 0));
end component;

component Adder3bit
Port ( A0 : in STD_LOGIC;
      A1 : in STD_LOGIC;
      A2 : in STD_LOGIC;
      B0 : in STD_LOGIC;
      B1 : in STD_LOGIC;
      B2 : in STD_LOGIC;
      S0 : out STD_LOGIC;
      S1 : out STD_LOGIC;
      S2 : out STD_LOGIC);
end component;

component Add_SubUnit
Port ( A0 : in STD_LOGIC;
      A1 : in STD_LOGIC;
      A2 : in STD_LOGIC;
      A3 : in STD_LOGIC;
      B0 : in STD_LOGIC;

```

```

        B1 : in STD_LOGIC;
        B2 : in STD_LOGIC;
        B3 : in STD_LOGIC;
        CTRL : in STD_LOGIC;
        S0 : inout STD_LOGIC;
        S1 : inout STD_LOGIC;
        S2 : inout STD_LOGIC;
        S3 : inout STD_LOGIC;
        overflow: out STD_LOGIC;
        zero: out STD_LOGIC);
end component;

component MUX8
Port ( Reg_S : in STD_LOGIC_VECTOR (2 downto 0);
      D0 : in STD_LOGIC_VECTOR (3 downto 0);
      D1 : in STD_LOGIC_VECTOR (3 downto 0);
      D2 : in STD_LOGIC_VECTOR (3 downto 0);
      D3 : in STD_LOGIC_VECTOR (3 downto 0);
      D4 : in STD_LOGIC_VECTOR (3 downto 0);
      D5 : in STD_LOGIC_VECTOR (3 downto 0);
      D6 : in STD_LOGIC_VECTOR (3 downto 0);
      D7 : in STD_LOGIC_VECTOR (3 downto 0);
      Mux8_4_out : out STD_LOGIC_VECTOR (3 downto 0));
end component;

component Slow_Clk
Port ( Clk_in : in STD_LOGIC;
      Clk_out : out STD_LOGIC);
end component;

component Mux2_4bit
Port (In_Adder_4: in STD_LOGIC_VECTOR (3 downto 0);
      In_Imm : in STD_LOGIC_VECTOR (3 downto 0);
      In_load_S : in STD_LOGIC;
      Mux2_4_out : out STD_LOGIC_VECTOR (3 downto 0));
end component;

component Mux2_3bit
Port ( In_jAddress : in STD_LOGIC_VECTOR (2 downto 0);
      In_Adder_3 : in STD_LOGIC_VECTOR (2 downto 0);
      JumpFlag : in STD_LOGIC;
      Mux2_3_out : out STD_LOGIC_VECTOR (2 downto 0));
end component;

component LUT_16_7New
Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
      data : out STD_LOGIC_VECTOR (6 downto 0));
end component;
signal EN_PC, Add_Sub_Sel, jmpflg, Load_Sel, EN_Store,S_Clk:
STD_LOGIC;
signal Data_Out_0 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_1 : STD_LOGIC_VECTOR(3 downto 0);

```

```

signal Data_Out_2 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_3 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_4 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_5 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_6 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_Out_7 : STD_LOGIC_VECTOR(3 downto 0);
signal Mem_Sel, Mux_2_3_out, Reg_Sel_A, Reg_Sel_B, jmpaddr:
STD_LOGIC_VECTOR(2 downto 0);
signal Mux_8_4_A_out, Mux_8_4_B_out : STD_LOGIC_VECTOR(3 downto 0);
signal Imm_Val : STD_LOGIC_VECTOR (3 downto 0);
signal Adder_4 : STD_LOGIC_VECTOR(3 downto 0);
signal Data_In : STD_LOGIC_VECTOR(3 downto 0);
signal Reg_Enable : STD_LOGIC_VECTOR(2 downto 0);
signal Instruction : STD_LOGIC_VECTOR(11 downto 0);
signal bit_3_out : STD_LOGIC_VECTOR(2 downto 0);
signal led_address : STD_LOGIC_VECTOR (3 downto 0);
signal led_data : STD_LOGIC_VECTOR (6 downto 0);

```

```

begin
Slow_Clk_0:Slow_Clk
port map( Clk_in=>Clk,
          Clk_out=>S_Clk
);
Instruction_Decoder_0 : InsDecoder
Port map ( I_Instruction => Instruction,
          I_Jump_Check => Mux_8_4_A_out,
          I_Clk => S_Clk,
          I_Reset=> Res,
          O_Reg_Enable => Reg_Enable,
          O_Reg_Sel_A => Reg_Sel_A,
          O_Reg_Sel_B => Reg_Sel_B,
          O_EN_PC => EN_PC,
          O_EN_Store => EN_Store,
          O_Add_Sub_Sel => Add_Sub_Sel,
          O_Jump_Flag => jmpflg,
          O_Jump_Addr =>jmpaddr ,
          O_Load_Sel => Load_Sel,
          O_Imm_Val => Imm_Val
);

```

```

Program_Counter_0 : PC
port map(
Res => Res,
Clk => S_Clk,
--EN=> EN_PC,
In_MUX2_3 => Mux_2_3_out,
Out_PC => Mem_Sel
);

```

```

Program_ROM_0 : PROM
Port map(
address_new => Mem_Sel,

```

```
data_new => Instruction
);
```

```
Reg_Bank_0 : RegBank
Port map (
I_Clk => S_Clk,
I_Reg_Enable => Reg_Enable,
I_Data_In => Data_In,
I_EN_Store => '1',
Data_out_0 => Data_Out_0,
Data_out_1 => Data_Out_1,
Data_out_2 => Data_Out_2,
Data_out_3 => Data_Out_3,
Data_out_4 => Data_Out_4,
Data_out_5 => Data_Out_5,
Data_out_6 => Data_Out_6,
Data_out_7 => Data_Out_7,
I_Reset => Res
);
```

```
Adder_3_0 : Adder3bit
Port map(A0=> Mem_Sel(0),
A1=> Mem_Sel(1),
A2=> Mem_Sel(2),
B0=>'1',
B1=>'0',
B2=>'0',
S0=>bit_3_out(0),
S1=>bit_3_out(1),
S2=>bit_3_out(2)
);
```

```
Add_Sub_4_0 : Add_SubUnit
Port map( A0=> Mux_8_4_A_out(0),
A1=> Mux_8_4_A_out(1),
A2=> Mux_8_4_A_out(2),
A3=> Mux_8_4_A_out(3),
B0 => Mux_8_4_B_out(0),
B1 => Mux_8_4_B_out(1),
B2 => Mux_8_4_B_out(2),
B3 => Mux_8_4_B_out(3),
CTRL => Add_Sub_Sel,
S0 => Adder_4(0),
S1 => Adder_4(1),
S2 => Adder_4(2),
S3 => Adder_4(3),
overflow => Overflow_led,
zero => Zero_led
);
```

```
MUX_8_4_A : MUX8
Port map( D7 => Data_Out_7 ,
```

```

D6 => Data_Out_6,
D5 => Data_Out_5,
D4 => Data_Out_4,
D3 => Data_Out_3,
D2 => Data_Out_2,
D1 => Data_Out_1,
D0 => Data_Out_0,
Reg_S => Reg_Sel_A,
Mux8_4_out => Mux_8_4_A_out
);
MUX_8_4_B : MUX8
Port map( D7 => Data_Out_7 ,
D6 => Data_Out_6,
D5 => Data_Out_5,
D4 => Data_Out_4,
D3 => Data_Out_3,
D2 => Data_Out_2,
D1 => Data_Out_1,
D0 => Data_Out_0,
Reg_S => Reg_Sel_B,
Mux8_4_out => Mux_8_4_B_out
);
MUX_2_4_0 : Mux2_4bit
Port map( In_Imm => Imm_Val,
In_Adder_4 => Adder_4,
In_load_S => Load_Sel,
MUX2_4_out=> Data_In
);

MUX_2_3_0 : Mux2_3bit
Port map( In_Adder_3 => bit_3_out,
In_jAddress=> jmpaddr ,
JumpFlag => jmpflg,
Mux2_3_out => Mux_2_3_out
);

LUT_16_7_0 : LUT_16_7New
port map(
address => led_address,
data => Seg7_Out
);

Reg_7<=Data_Out_7;
--Reg_1<=Data_Out_0;
led_address <= Data_Out_7;
Seg7_Anode <= "1110";

end Behavioral;

```

Program Counter

```

entity PC is
    Port ( In_MUX2_3 : in STD_LOGIC_VECTOR (2 downto 0);
          --EN : in STD_LOGIC;
          Clk : in STD_LOGIC;
          --Inc: in STD_LOGIC;
          Res: in STD_LOGIC;
          Out_PC : inout STD_LOGIC_VECTOR (2 downto 0));
end PC;

architecture Behavioral of PC is

begin
    process (Clk) begin
        if (rising_edge(Clk)) then
            if Res = '1' then
                Out_PC <= "000";
            else
                --if EN = '1' then
                    Out_PC <= In_MUX2_3;
                --end if;
            end if;
        end if ;
    end process;
end Behavioral;

```

Program Rom

```

entity PROM is
    Port ( address_new : in STD_LOGIC_VECTOR (2 downto 0);
          data_new : out STD_LOGIC_VECTOR (11 downto 0));
end PROM;

architecture Behavioral of PROM is
    type rom_type is array (0 to 7) of std_logic_vector(11 downto 0);
    signal assemblyInstructions_ROM : rom_type := (
        --"100010001010", -- MOVI R1, 10 ; R1 ? 10
        --"100100000001",--MOVI R2, 1 ; R2 ? 1
        --"010100000000",-- NEG R2 ; R2 ? -R2
        --"000010100000",-- ADD R1, R2 ; R1 ? R1 + R2
        --"110010000111",-- JZR R1, 7 ; If R1 = 0 jump to line 7
        --"110000000011"--JZR R0, 3 ; If R0 = 0 jump to line 3
        "101110000001", --0 Move 1 to R7
        "100100000010", --1 Move 2 to R2
        "100110000011", --2 Move 3 to R3
        "001110100000", --3 R7 <= R7 + R2
    );
end Behavioral;

```

```

        "001110110000", --4 R7 <= R7 + R3
        "010110000000", --5 R3 <= -R3
        "111110000101", --6 Jump to instruction 5 if Reg7 is 0
        "110000000110"  --7 Jump to instruction 6 if Reg0 is 0
    );

begin
data_new <=
assemblyInstructions_ROM(to_integer(unsigned(address_new)));

end Behavioral;

```

Reg Bank

```

entity RegBank is
    Port (
        I_Reg_Enable : in STD_LOGIC_VECTOR (2 downto 0);
        I_Data_In : in STD_LOGIC_VECTOR (3 downto 0);
        I_Reset : in STD_LOGIC;
        I_En_Store : in STD_LOGIC;
        I_Clk : in STD_LOGIC;
        --Data_out : out STD_LOGIC_VECTOR (31 downto 0)
        Data_out_0:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_1:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_2:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_3:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_4:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_5:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_6:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_7:out STD_LOGIC_VECTOR (3 downto 0)

    );
end RegBank;

```

```

architecture Behavioral of RegBank is
    component Decoder3New is
        Port (
            I : in STD_LOGIC_VECTOR (2 downto 0);
            EN : in STD_LOGIC;
            Y : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    component Register4 is
        Port (
            D : in STD_LOGIC_VECTOR (3 downto 0);
            En : in STD_LOGIC;
            En_Store : in STD_LOGIC;

```

```

        Res : in STD_LOGIC;
        Clk : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR (3 downto 0)
    );
end component;

signal Reg_Sel : STD_LOGIC_VECTOR(7 downto 0);

begin
    Decoder3_8_0: Decoder3New
    port map (
        I => I_Reg_Enable,
        EN => I_En_Store,
        Y => Reg_Sel
    );

    --Data_out(31 downto 28) <= "0000";
    Data_out_0 <= "0000";

    Reg_1: Register4
    port map (
        D => I_Data_In,
        En => Reg_Sel(1),
        En_Store => I_En_Store,
        Res => I_Reset,
        Clk => I_Clk,
        Q => Data_out_1
    );

    Reg_2: Register4
    port map (
        D => I_Data_In,
        En => Reg_Sel(2),
        En_Store => I_En_Store,
        Res => I_Reset,
        Clk => I_Clk,
        Q => Data_out_2
    );

    Reg_3: Register4
    port map (
        D => I_Data_In,
        En => Reg_Sel(3),
        En_Store => I_En_Store,
        Res => I_Reset,
        Clk => I_Clk,
        Q => Data_out_3
    );

    Reg_4: Register4
    port map (
        D => I_Data_In,

```

```

        En => Reg_Sel(4),
        En_Store => I_En_Store,
        Res => I_Reset,
        Clk => I_Clk,
        Q => Data_out_4
    );

Reg_5: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(5),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_5
);

Reg_6: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(6),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_6
);

Reg_7: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(7),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_7
);

end Behavioral;

```

Reg

```

entity RegBank is
    Port (
        I_Reg_Enable : in STD_LOGIC_VECTOR (2 downto 0);
        I_Data_In : in STD_LOGIC_VECTOR (3 downto 0);
        I_Reset : in STD_LOGIC;
        I_En_Store : in STD_LOGIC;
        I_Clk : in STD_LOGIC;
    );
end entity;

```

```

        --Data_out : out STD_LOGIC_VECTOR (31 downto 0)
        Data_out_0:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_1:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_2:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_3:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_4:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_5:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_6:out STD_LOGIC_VECTOR (3 downto 0);
        Data_out_7:out STD_LOGIC_VECTOR (3 downto 0)

    );
end RegBank;

architecture Behavioral of RegBank is
    component Decoder3New is
        Port (
            I : in STD_LOGIC_VECTOR (2 downto 0);
            EN : in STD_LOGIC;
            Y : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    component Register4 is
        Port (
            D : in STD_LOGIC_VECTOR (3 downto 0);
            En : in STD_LOGIC;
            En_Store : in STD_LOGIC;
            Res : in STD_LOGIC;
            Clk : in STD_LOGIC;
            Q : out STD_LOGIC_VECTOR (3 downto 0)
        );
    end component;

    signal Reg_Sel : STD_LOGIC_VECTOR(7 downto 0);

begin
    Decoder3_8_0: Decoder3New
    port map (
        I => I_Reg_Enable,
        EN => I_En_Store,
        Y => Reg_Sel
    );

    --Data_out(31 downto 28) <= "0000";
    Data_out_0 <= "0000";

    Reg_1: Register4
    port map (
        D => I_Data_In,
        En => Reg_Sel(1),
        En_Store => I_En_Store,
        Res => I_Reset,

```

```

        Clk => I_Clk,
        Q => Data_out_1
    );

Reg_2: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(2),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_2
);

Reg_3: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(3),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_3
);

Reg_4: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(4),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_4
);

Reg_5: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(5),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_5
);

Reg_6: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(6),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_6

```

```

);

Reg_7: Register4
port map (
    D => I_Data_In,
    En => Reg_Sel(7),
    En_Store => I_En_Store,
    Res => I_Reset,
    Clk => I_Clk,
    Q => Data_out_7
);

end Behavioral;

```

Decoder 3 to 8

```

entity Decoder3New is
    Port ( I : in STD_LOGIC_VECTOR (2 downto 0);
          EN : in STD_LOGIC;
          Y : out STD_LOGIC_VECTOR (7 downto 0));
end Decoder3New;

```

```

architecture Behavioral of Decoder3New is
    component Decoder2_4
        port(
            I: in STD_LOGIC_VECTOR;
            EN: in STD_LOGIC;
            Y: out STD_LOGIC_VECTOR );
    end component;
    signal I0,I1 : STD_LOGIC_VECTOR (1 downto 0);
    signal Y0,Y1 : STD_LOGIC_VECTOR (3 downto 0);
    signal en0,en1, I2 : STD_LOGIC;

```

```

begin
    Decode2_4_0 : Decoder2_4
        port map(
            I => I0,
            EN => en0,
            Y => Y0 );
    Decode2_4_1 : Decoder2_4
        port map(
            I => I1,
            EN => en1,
            Y => Y1 );
    en0 <= NOT(I(2)) AND EN;
    en1 <= I(2) AND EN;
    I0 <= I(1 downto 0);
    I1 <= I(1 downto 0);

```

```
I2 <= I(2);
Y(3 downto 0) <= Y0;
Y(7 downto 4) <= Y1;
```

```
end Behavioral;
```

Decoder 2 to 4

```
entity Decoder2_4 is
    Port ( I : in STD_LOGIC_VECTOR (1 downto 0);
          EN : in STD_LOGIC;
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end Decoder2_4;
```

```
architecture Behavioral of Decoder2_4 is
```

```
begin
process (I, EN)
begin
    if (EN='1') then -- active high enable pin
        case I is
            when "00" => Y <= "0001";
            when "01" => Y <= "0010";
            when "10" => Y <= "0100";
            when "11" => Y <= "1000";
            when others => Y <= "0000";
        end case;
    else
        Y<="0000";
    end if;
end process;
```

```
end Behavioral;
```

Add Sub 4

```
entity Add_SubUnit is
    Port ( A0 : in STD_LOGIC;
          A1 : in STD_LOGIC;
          A2 : in STD_LOGIC;
          A3 : in STD_LOGIC;
          B0 : in STD_LOGIC;
```

```

        B1 : in STD_LOGIC;
        B2 : in STD_LOGIC;
        B3 : in STD_LOGIC;
        CTRL : in STD_LOGIC;
        S0 : inout STD_LOGIC;
        S1 : inout STD_LOGIC;
        S2 : inout STD_LOGIC;
        S3 : inout STD_LOGIC;
        overflow: out STD_LOGIC;
        zero: out STD_LOGIC);
        --C_out : out STD_LOGIC);
end Add_SubUnit;

architecture Behavioral of Add_SubUnit is
    component FAUnit
    port (
        A: in std_logic;
        B: in std_logic;
        C_in: in std_logic;
        S: out std_logic;
        C_out: out std_logic);
    end component;
    SIGNAL  FA0_C, FA1_C, FA2_C, FA3_C,B0_S,B1_S,B2_S,B3_S: std_logic;

begin

    B0_S<=B0 XOR CTRL;
    B1_S<=B1 XOR CTRL;
    B2_S<=B2 XOR CTRL;
    B3_S<=B3 XOR CTRL;

    FA_0 : FAUnit
    port map (
        A => A0,
        B =>B0_S,
        C_in => CTRL, -- Set to ground C_in => C_in,
        S => S0,
        C_out => FA0_C);
    FA_1 : FAUnit
    port map (
        A => A1,
        B => B1_S,
        C_in => FA0_C,
        S => S1,
        C_out => FA1_C);
    FA_2 : FAUnit
    port map (
        A => A2,
        B => B2_S,
        C_in => FA1_C,
        S => S2,
        C_out => FA2_C);

```

```

FA_3 : FAUnit
port map (
A => A3,
B => B3_S,
C_in => FA2_C,
S => S3,
C_out => overflow);
zero<= not (S0 or S1 or S2 or S3);

end Behavioral;

```

FA

```

entity FAUnit is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           C_in : in STD_LOGIC;
           S : out STD_LOGIC;
           C_out : out STD_LOGIC);
end FAUnit;

architecture Behavioral of FAUnit is
    component HA
    port (
        A: in std_logic;
        B: in std_logic;
        S: out std_logic;
        C: out std_logic);
    end component;
    SIGNAL HA0_S, HA0_C, HA1_S, HA1_C : std_logic;
begin
    HA_0 : HA
    port map (
        A => A,
        B => B,
        S => HA0_S,
        C => HA0_C);
    HA_1 : HA
    port map (
        A => HA0_S,
        B => C_in,
        S => HA1_S,
        C => HA1_C);
    S <= HA1_S;
    C_out <= HA0_C OR HA1_C;

end Behavioral;

```

HA

```
entity HA is
    Port ( A : in STD_LOGIC;
           B : in STD_LOGIC;
           C : out STD_LOGIC;
           S : out STD_LOGIC);
end HA;

architecture Behavioral of HA is

begin
    S<=A XOR B;
    C<= A AND B;

end Behavioral;
```

Adder_3

```
entity Adder3bit is
    Port ( A0 : in STD_LOGIC;
           A1 : in STD_LOGIC;
           A2 : in STD_LOGIC;
           B0 : in STD_LOGIC;
           B1 : in STD_LOGIC;
           B2 : in STD_LOGIC;
           S0 : out STD_LOGIC;
           S1 : out STD_LOGIC;
           S2 : out STD_LOGIC);
end Adder3bit;

architecture Behavioral of Adder3bit is
    component FAUnit
    port (
        A: in std_logic;
        B: in std_logic;
        C_in: in std_logic;
        S: out std_logic;
        C_out: out std_logic);
    end component;
    SIGNAL  FA0_C, FA1_C, FA2_C : std_logic;
```

```

begin

FA_0 : FAUnit
port map (
A => A0,
B => B0,
C_in => '0', -- Set to ground C_in => C_in,
S => S0,
C_out => FA0_C);
FA_1 : FAUnit
port map (
A => A1,
B => B1,
C_in => FA0_C,
S => S1,
C_out => FA1_C);
FA_2 : FAUnit
port map (
A => A2,
B => B2,
C_in => FA1_C,
S => S2,
C_out => FA2_C);

end Behavioral;

```

Instruction Decoder

```

entity InsDecoder is
    Port (
        I_Instruction : in STD_LOGIC_VECTOR (11 downto 0);
        I_Jump_Check : in STD_LOGIC_VECTOR (3 downto 0);
        I_Clk : in STD_LOGIC;
        I_Reset : in STD_LOGIC;
        O_Reg_Enable : out STD_LOGIC_VECTOR (2 downto 0);
        O_Reg_Sel_A : out STD_LOGIC_VECTOR (2 downto 0);
        O_Reg_Sel_B : out STD_LOGIC_VECTOR (2 downto 0);
        O_EN_PC : out STD_LOGIC;
        O_EN_Store : out STD_LOGIC;
        O_Add_Sub_Sel : out STD_LOGIC;
        O_Jump_Flag : out STD_LOGIC;
        O_Jump_Addr : out STD_LOGIC_VECTOR (2 downto 0);
        O_Load_Sel : out STD_LOGIC;
        O_Imm_Val : out STD_LOGIC_VECTOR (3 downto 0)
    );
end InsDecoder;

```

```
architecture Behavioral of InsDecoder is
SIGNAL reg_access: STD_LOGIC;
```

```
begin
```

```
    O_Load_Sel <= NOT(I_Instruction(11));
    --O_Reg_Enable <= I_Instruction(9 downto 7);
reg_access <= NOT (I_Instruction(11) AND I_Instruction(10));
    O_Reg_Sel_A <= I_Instruction(9 downto 7);
    O_Reg_Sel_B <= I_Instruction(6 downto 4);
    O_Jump_Addr <= I_Instruction(2 downto 0);

    O_Jump_Flag <= '1' when (I_Instruction(11) = '1' and
I_Instruction(10) = '1' and I_Jump_Check = "0000") else '0';
    O_Reg_Enable <= I_Instruction(9 downto 7) when reg_access = '1'
else "000";
    O_Add_Sub_Sel <= '1' when (NOT I_Instruction(11) = '1' and
I_Instruction(10) = '1') else '0';
    O_Imm_Val <= I_Instruction(3 downto 0);

    O_EN_Store <= '1' when (I_Instruction(11) = '0' and
I_Instruction(10) = '0') else '0';
    O_EN_PC <= '1' when (I_Instruction(11) = '1' and I_Instruction(10)
= '0') else '0';

end Behavioral;
```

MUX2_3

```
entity Mux2_3bit is
```

```
    Port ( In_jAddress : in STD_LOGIC_VECTOR (2 downto 0);
          In_Adder_3 : in STD_LOGIC_VECTOR (2 downto 0);
          JumpFlag : in STD_LOGIC;
          Mux2_3_out : out STD_LOGIC_VECTOR (2 downto 0));
end Mux2_3bit ;
```

```
architecture Behavioral of Mux2_3bit is
```

```
begin
```

```
process(JumpFlag, In_jAddress, In_Adder_3)
```

```
begin
```

```
Mux2_3_out(0) <= (NOT(JumpFlag) AND In_Adder_3(0)) OR (JumpFlag AND
In_jAddress(0));
```

```
Mux2_3_out(1) <= (NOT(JumpFlag) AND In_Adder_3(1)) OR (JumpFlag AND
In_jAddress(1));
```

```

Mux2_3_out(2) <= (NOT(JumpFlag) AND In_Adder_3(2)) OR (JumpFlag AND
In_jAddress(2));
end process;

end Behavioral;

```

MUX 2_4

```

entity Mux2_4bit is
    Port (In_Adder_4: in STD_LOGIC_VECTOR (3 downto 0);
          In_Imm : in STD_LOGIC_VECTOR (3 downto 0);
          In_load_S : in STD_LOGIC;
          Mux2_4_out : out STD_LOGIC_VECTOR (3 downto 0));
end Mux2_4bit;

architecture Behavioral of Mux2_4bit is

begin
    process(In_load_S, In_Adder_4, In_Imm)
    begin
        --if In_load_S='0' then
        --Mux2_4_out<=In_Adder_4;
        --else
        --Mux2_4_out<=In_Imm;
        --end if;
        Mux2_4_out(0) <= (NOT(In_load_S) AND In_Imm(0)) OR (In_load_S AND
In_Adder_4(0));
        Mux2_4_out(1) <= (NOT(In_load_S) AND In_Imm(1)) OR (In_load_S AND
In_Adder_4(1));
        Mux2_4_out(2) <= (NOT(In_load_S) AND In_Imm(2)) OR (In_load_S AND
In_Adder_4(2));
        Mux2_4_out(3) <= (NOT(In_load_S) AND In_Imm(3)) OR (In_load_S AND
In_Adder_4(3));
    end process;

end Behavioral;

```

MUX 8

```

entity MUX8 is
    Port ( Reg_S : in STD_LOGIC_VECTOR (2 downto 0);
          D0 : in STD_LOGIC_VECTOR (3 downto 0);
          D1 : in STD_LOGIC_VECTOR (3 downto 0);

```

```

        D2 : in STD_LOGIC_VECTOR (3 downto 0);
        D3 : in STD_LOGIC_VECTOR (3 downto 0);
        D4 : in STD_LOGIC_VECTOR (3 downto 0);
        D5 : in STD_LOGIC_VECTOR (3 downto 0);
        D6 : in STD_LOGIC_VECTOR (3 downto 0);
        D7 : in STD_LOGIC_VECTOR (3 downto 0);
        Mux8_4_out : out STD_LOGIC_VECTOR (3 downto 0));
end MUX8;

```

architecture Behavioral of MUX8 is

```

begin
process (Reg_S, D0, D1, D2, D3, D4, D5, D6, D7)
begin
    case Reg_S is
        when "000" =>
            Mux8_4_out <= D0;
        when "001" =>
            Mux8_4_out <= D1;
        when "010" =>
            Mux8_4_out <= D2;
        when "011" =>
            Mux8_4_out <= D3;
        when "100" =>
            Mux8_4_out <= D4;
        when "101" =>
            Mux8_4_out <= D5;
        when "110" =>
            Mux8_4_out <= D6;
        when "111" =>
            Mux8_4_out <= D7;
        when others =>
            Mux8_4_out <= (others => 'U');
    end case;
end process;

```

end Behavioral;

LUT 16_7

```

entity LUT_16_7New is
    Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
          data : out STD_LOGIC_VECTOR (6 downto 0));
end LUT_16_7New;

```

architecture Behavioral of LUT_16_7New is

```

type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
signal sevenSegment_ROM : rom_type := (
    "1000000", -- 0
    "1111001", --1
    "0100100", --2
    "0110000", --3
    "0011001", --4
    "0010010", --5
    "0000010", --6
    "1111000", --7
    "0000000", --8
    "0010000", --9
    "0001000", -- a
    "0000011", --b
    "1000110", --c
    "0100001", --d
    "0000110", --e
    "0001110" -- f
);

begin
data <= sevenSegment_ROM(to_integer(unsigned(address)));

end Behavioral;

```

Slow Clock

```

entity Slow_Clk is
    Port ( Clk_in : in STD_LOGIC;
           Clk_out : out STD_LOGIC);
end Slow_Clk;

architecture Behavioral of Slow_Clk is
    signal count: integer :=1;
    signal clk_status: std_logic :='0';

begin
    --For 100MHz input clock this generates 1Hz clock
    process (Clk_in) begin
        if (rising_edge(Clk_in)) then
            count<=count+1;
            if (count=50000000) then--0000000
                --if(count =4) then
                    clk_status<=not clk_status;
                    Clk_out<=clk_status;
                    count<=1;
                end if;
            end if;
        end if;
    end process;
end Behavioral;

```

```

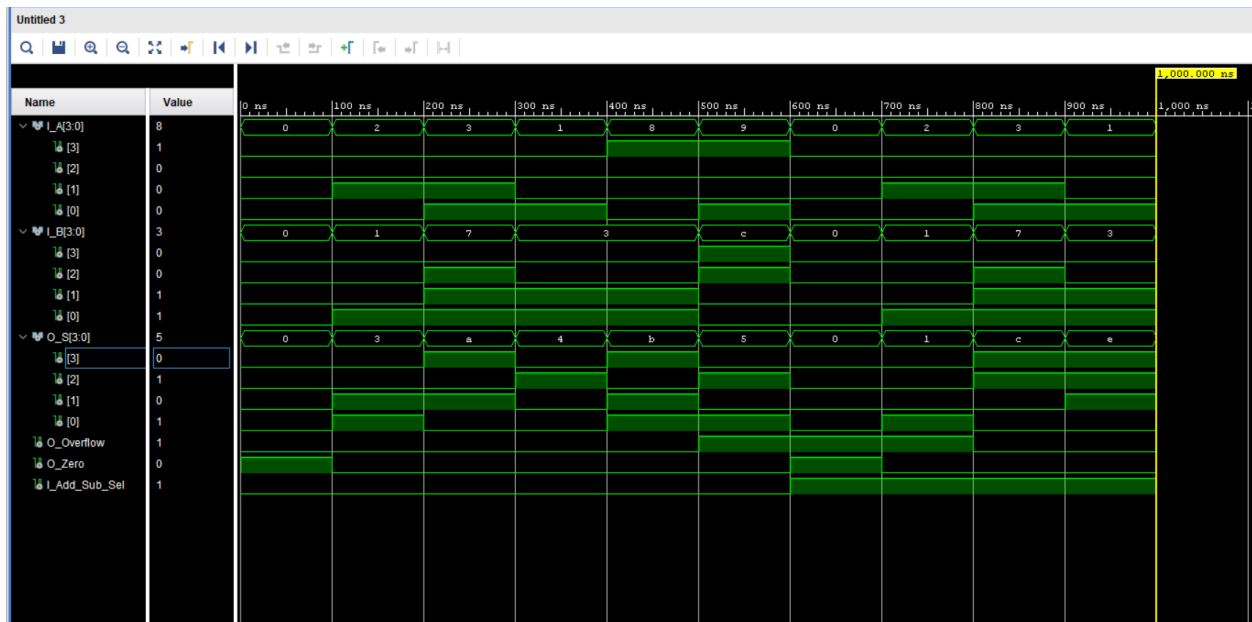
        end if;
    end if;
end process;

end Behavioral;

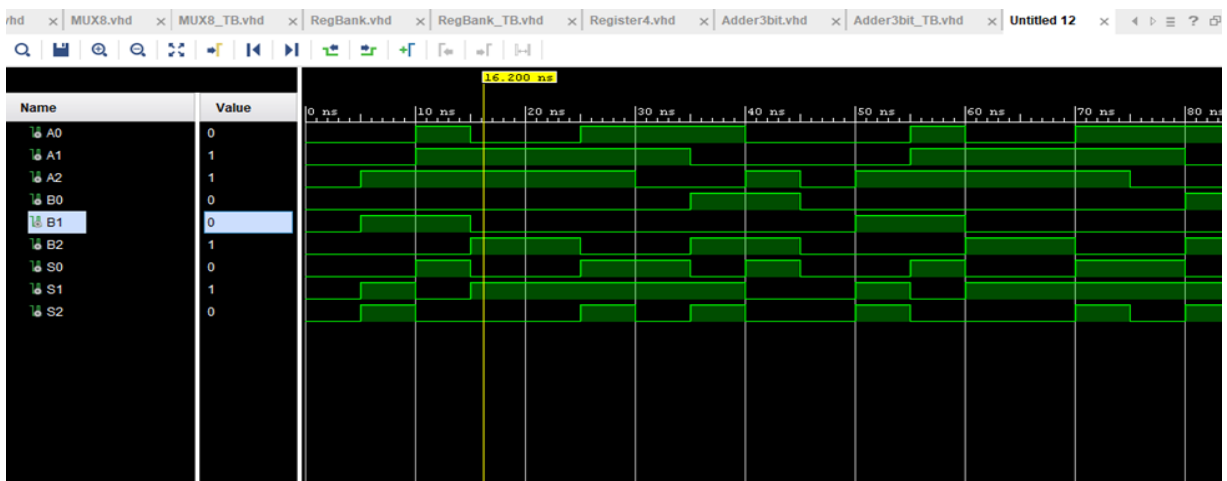
```

Timing Diagrams

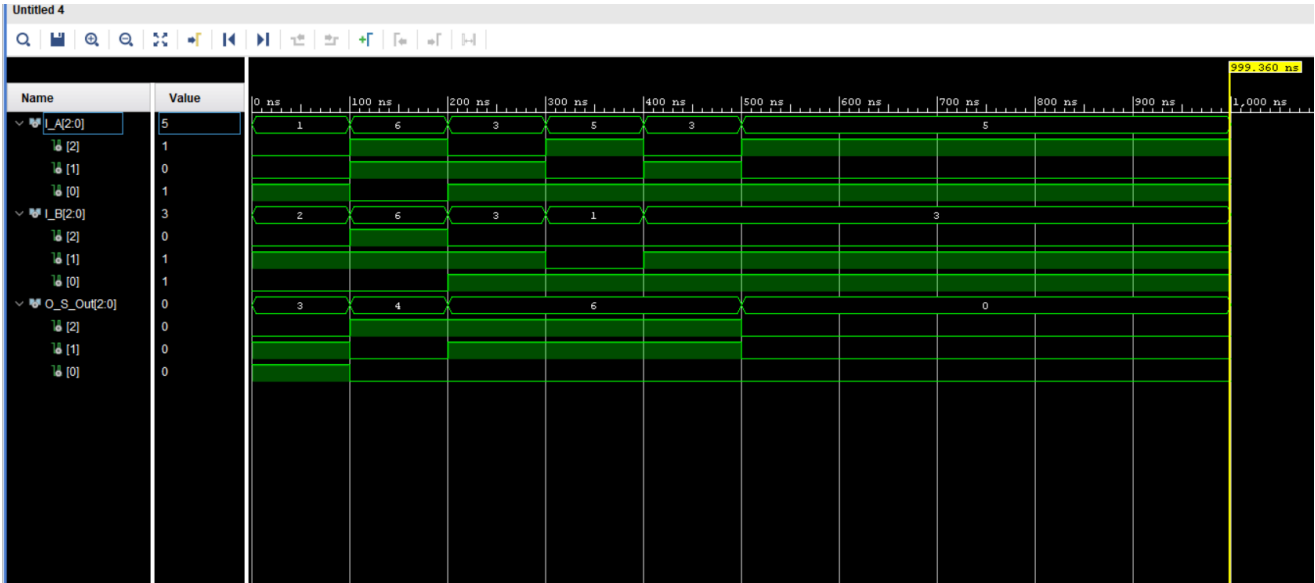
Add_Sub_4



Adder_3



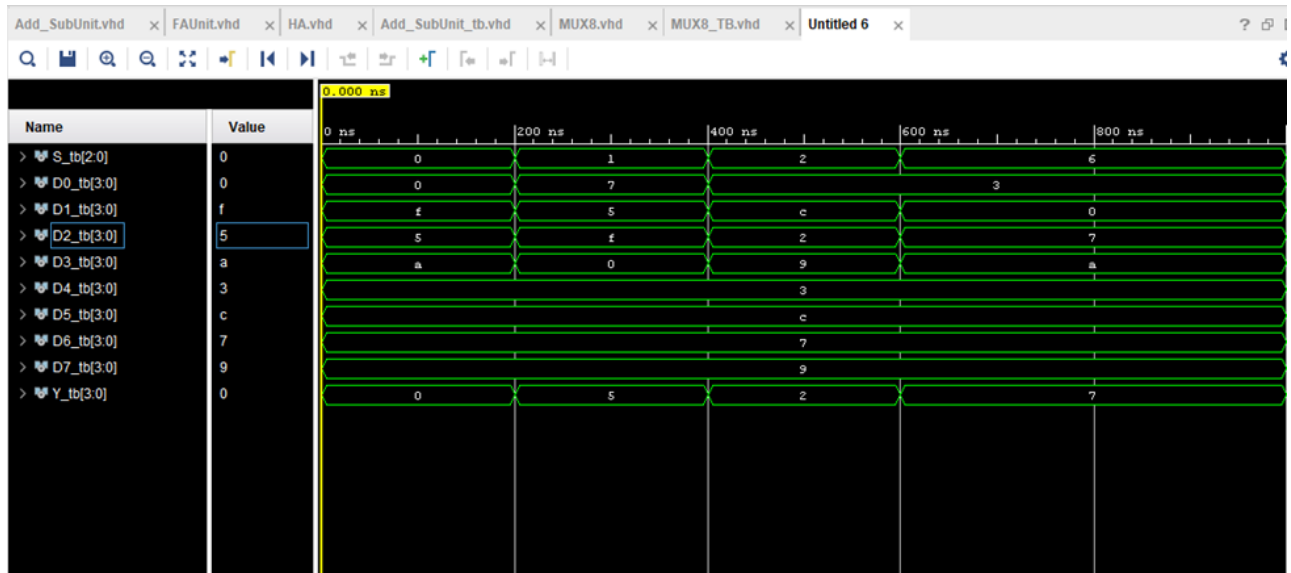
2 Way 3 Bit Multiplexer



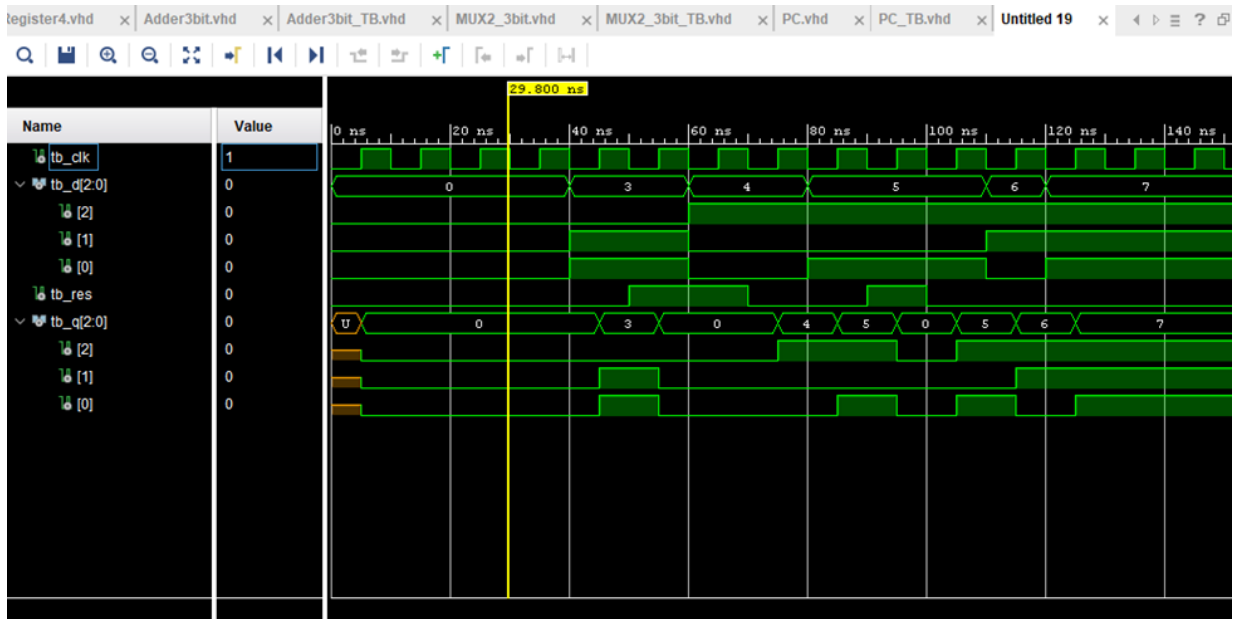
2 Way 4 Bit Multiplexer



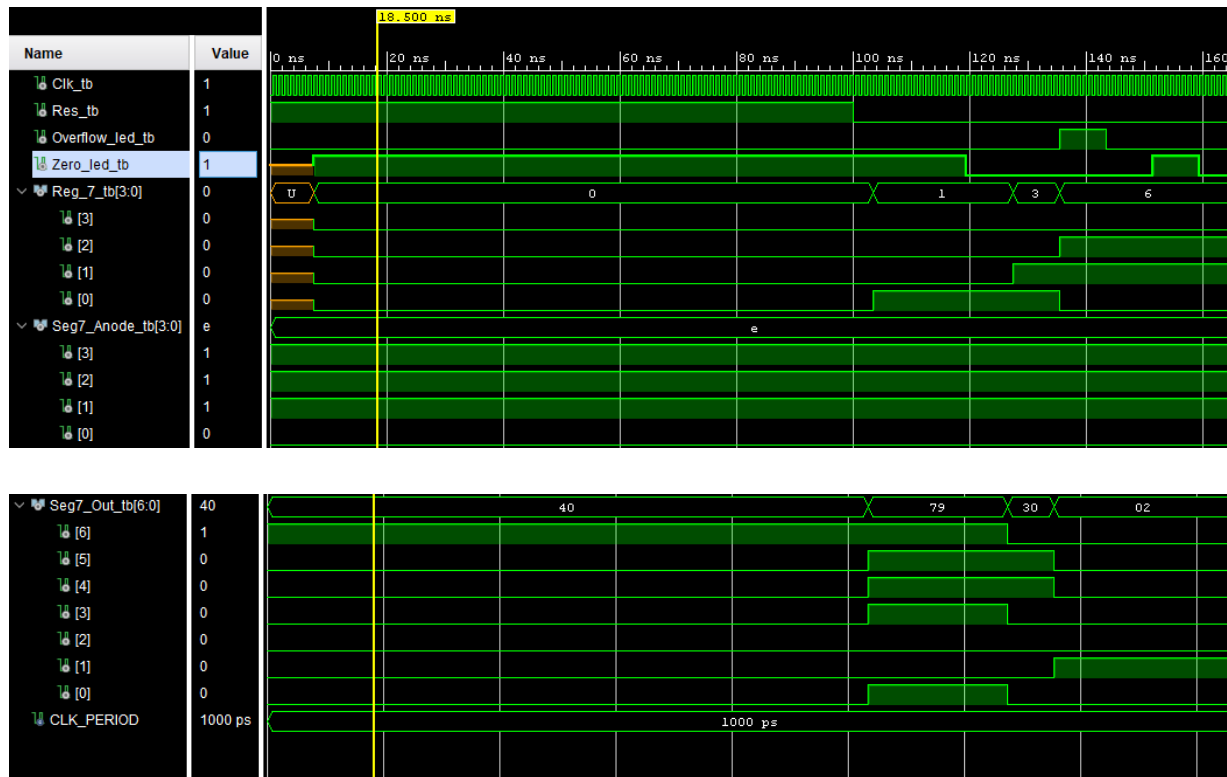
8 Way 4 Bit Multiplexer



Program Counter



Nanoprocessor



Conclusions

- We leveraged many of the components we had previously created in our previous labs, allowing us to save time and resources.
- Developing different components independently and then integrating them together proved to be a faster approach, as it allowed different team members to work on different components simultaneously.
- The utilization of large registers or combinations of flip-flops significantly increased the logical unit (LUT) usage
- The simulator proved to be extremely valuable for debugging purposes once we had all the components connected and functioning together.

Contributions

Name	Task	Hours contributed
Mallikarachchi S.D	Program ROM Program Counter Instruction Decoder Register Bank Component Mapping Debug Coding	13
Wickramasinghe J.J	3-bit adder 4-bit add-sub unit 2 way 3 bit multiplexer 2 way 4 bit multiplexer 8 way 4 bit multiplexer	10